

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240002 123240017
NAMA	:	Fahri Hidayatullah Gian Abi Firdaus
PLUG	:	IF-J
NAMA ASISTEN	:	Bagas Duta Prasetya Nandisya Faiz Effendi

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Fahri Hidayatullah

123240002

Gian Abi Firdaus

123240017

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan
Pada Tanggal: 4 Juni 2026

Asisten Praktikum

Asisten Praktikum

Bagas Duta Prasetya
NIM. 123220057

Nandisya Faiz Effendi
NIM. 123220139

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul **Pemilihan Jadwal Kereta Api Terbaik Dari St. Pasar Senen (PSE) Menuju St. Yogyakarta (YK) Atau St. Lempuyangan (LPN)**. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 4 Juni 2026

Penyusun 1,

Penyusun 2,

Fahri Hidayatullah
NIM. 123240002

Gian Abi Firdaus
NIM. 123240017

DAFTAR ISI

HALAMAN PENGESAHAN	ii
KATA PENGANTAR.....	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Tujuan Proyek Akhir	1
1.3 Manfaat Proyek Akhir	2
BAB II PEMBAHASAN.....	3
2.1 Dasar Teori	3
2.2 Skenario Kasus & Dataset	5
2.3 Pemodelan Sistem Pendukung Keputusan	7
2.4 Perhitungan & Algoritma	11
2.5 Implementasi & Tampilan Sistem	13
BAB III JADWAL Pengerjaan dan Pembagian Tugas	35
3.1 Jadwal Pengerjaan	35
3.2 Pembagian Tugas.....	35
BAB IV KESIMPULAN DAN SARAN.....	37
4.2 Kesimpulan.....	37
4.3 Saran	37
DAFTAR PUSTAKA.....	39

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Transportasi kereta api menjadi salah satu pilihan utama masyarakat mobilitas tinggi untuk bepergian dari Jakarta menuju Yogyakarta karena dinilai aman, nyaman, dan relatif tepat waktu. Stasiun Pasar Senen (PSE) di Jakarta merupakan salah satu titik keberangkatan terpadat untuk kelas ekonomi dan eksekutif, dengan tujuan akhir di wilayah Yogyakarta yang umumnya dilayani oleh Stasiun Yogyakarta (YK) dan Stasiun Lempuyangan (LPN).

Banyaknya variasi jadwal keberangkatan, durasi perjalanan, kelas kereta (ekonomi, eksekutif), hingga variasi harga tiket sering kali membuat calon penumpang mengalami kesulitan dalam menentukan pilihan terbaik. Setiap penumpang memiliki prioritas yang berbeda-beda; sebagian mengutamakan harga tiket yang paling murah, sebagian mengejar efisiensi waktu perjalanan, dan sebagian lainnya memprioritaskan kenyamanan fasilitas kelas kereta atau ketepatan waktu tiba.

Proses pemilihan manual yang dilakukan dengan membandingkan satu per satu jadwal di aplikasi pemesanan tiket sering kali tidak efisien dan kurang objektif. Oleh karena itu, diperlukan sebuah Sistem Pendukung Keputusan (SPK) yang mampu mengolah berbagai kriteria tersebut. Dengan menerapkan metode kualitatif dan kuantitatif dari Sistem Cerdas, sistem ini dapat memberikan rekomendasi jadwal kereta api yang paling optimal secara cepat dan akurat sesuai dengan preferensi masing-masing pengguna

1.2 Tujuan Proyek Akhir

Adapun tujuan yang ingin dicapai melalui pengerjaan proyek akhir ini adalah:

- Membangun sebuah Sistem Pendukung Keputusan (SPK) yang dapat membantu pengguna menentukan pilihan jadwal kereta api terbaik dari Stasiun Pasar Senen (PSE) menuju Stasiun Yogyakarta (YK) atau Stasiun Lempuyangan (LPN).

- Menerapkan metode Sistem Pendukung Keputusan AHP dalam melakukan pembobotan kriteria dan perankingan alternatif jadwal kereta api secara objektif.
- Mempermudah calon penumpang dalam menganalisis alternatif pilihan berdasarkan kombinasi kriteria harga, waktu tempuh, waktu tiba, jenis rangkaian, dan stasiun pemberhentian.

1.3 Manfaat Proyek Akhir

Penyusunan dan implementasi sistem ini diharapkan dapat memberikan keuntungan dan manfaat bagi berbagai pihak, antara lain:

- **Bagi Pengguna/Calon Penumpang:** Mempercepat proses pengambilan keputusan dalam memilih jadwal kereta tanpa harus membandingkan data secara manual, serta mendapatkan rekomendasi yang paling sesuai dengan prioritas kebutuhan (budget-oriented atau time-oriented).
- **Bagi Penulis:** Sebagai sarana untuk mengimplementasikan teori dan materi perkuliahan Praktikum Sistem Cerdas dan Pendukung Keputusan ke dalam sebuah studi kasus nyata di dunia transportasi.
- **Bagi Pengembangan Sistem:** Menjadi referensi dasar atau framework dalam pengembangan sistem rekomendasi transportasi massal yang lebih luas di masa mendatang.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Analytical Hierarchy Process (AHP) merupakan salah satu metode *Multi-Criteria Decision Making* (MCDM) yang digunakan untuk membantu pengambilan keputusan pada permasalahan yang melibatkan banyak kriteria. Metode ini memungkinkan pengambil keputusan untuk menguraikan suatu masalah ke dalam struktur hierarki yang terdiri atas tujuan, kriteria, dan alternatif sehingga proses evaluasi dapat dilakukan secara lebih sistematis. Menurut Ashour dan Mahdiyar (2024), AHP merupakan metode yang efektif dalam menangani keputusan kompleks karena mampu menggabungkan penilaian kuantitatif dan kualitatif serta menghasilkan prioritas alternatif yang jelas (Ashour & Mahdiyar, 2024). Oleh karena itu, AHP banyak digunakan dalam berbagai bidang, termasuk transportasi, manajemen, pendidikan, dan sistem pendukung keputusan.

Pada sistem pendukung keputusan pemilihan kereta api yang dikembangkan, AHP digunakan untuk menentukan alternatif kereta terbaik berdasarkan lima kriteria, yaitu biaya, jenis rangkaian, waktu sampai, waktu tempuh, dan jumlah pemberhentian. Tahap pertama dalam metode AHP adalah membentuk matriks perbandingan berpasangan (*pairwise comparison matrix*). Pengguna membandingkan tingkat kepentingan antar kriteria menggunakan skala Saaty 1–9. Menurut Madzík dan Falát (2022), perbandingan berpasangan merupakan inti dari AHP karena memungkinkan pengambil keputusan menilai tingkat kepentingan relatif antar kriteria secara lebih mudah dibandingkan memberikan bobot secara langsung (Madzík & Falát, 2022). Pada program yang dibuat, proses ini terlihat pada pengisian matriks perbandingan berpasangan yang kemudian digunakan untuk menghitung bobot kriteria.

Setelah matriks perbandingan terbentuk, dilakukan proses normalisasi untuk mengubah nilai-nilai dalam matriks ke dalam skala yang sebanding. Selanjutnya, bobot prioritas diperoleh dengan menghitung rata-rata setiap baris dari matriks yang telah dinormalisasi. Kumar dan Pant (2022) menjelaskan bahwa bobot prioritas yang

dihasilkan menunjukkan tingkat kontribusi masing-masing kriteria terhadap tujuan pengambilan keputusan sehingga dapat digunakan sebagai dasar dalam mengevaluasi alternatif yang tersedia (Kumar & Pant, 2022). Pada sistem ini, perhitungan tersebut diimplementasikan melalui fungsi `hitung_vektor_prioritas()` yang menghasilkan bobot setiap kriteria berdasarkan hasil normalisasi matriks.

Salah satu keunggulan utama AHP adalah kemampuannya dalam mengukur konsistensi penilaian pengambil keputusan. Pengujian konsistensi dilakukan menggunakan Consistency Index (CI) dan Consistency Ratio (CR). Menurut Salehzadeh dan Ziaean (2024), pengukuran konsistensi sangat penting karena dapat memastikan bahwa penilaian yang diberikan tidak bersifat kontradiktif dan tetap logis. Suatu matriks perbandingan dianggap memiliki tingkat konsistensi yang baik apabila nilai $CR \leq 0,10$ (Salehzadeh & Ziaean, 2024). Pada program ini, apabila nilai CR melebihi 0,10 maka pengguna diminta untuk memperbaiki nilai perbandingan yang telah diberikan sebelum melanjutkan ke tahap berikutnya.

Selain digunakan untuk menentukan bobot kriteria, metode AHP pada sistem ini juga diterapkan untuk menghitung prioritas alternatif kereta berdasarkan masing-masing kriteria. Sistem membentuk matriks perbandingan alternatif menggunakan data aktual seperti harga tiket, jenis rangkaian, waktu kedatangan, durasi perjalanan, dan jumlah pemberhentian. Untuk kriteria bertipe *cost* seperti harga dan waktu tempuh, nilai yang lebih rendah dianggap lebih baik, sedangkan untuk kriteria bertipe *benefit* seperti kualitas jenis rangkaian, nilai yang lebih tinggi dianggap lebih baik. Menurut Kovačić dkk. (2024), penerapan AHP dalam bidang transportasi sangat efektif karena mampu mengakomodasi berbagai kriteria yang memiliki karakteristik berbeda dalam satu proses pengambilan keputusan yang terintegrasi (Kovačić et al., 2024).

Tahap terakhir dalam metode AHP adalah sintesis global, yaitu menggabungkan bobot kriteria dengan bobot alternatif untuk memperoleh skor akhir setiap alternatif. Skor akhir dihitung melalui penjumlahan hasil perkalian antara bobot kriteria dan bobot alternatif pada masing-masing kriteria. Alternatif dengan skor tertinggi akan direkomendasikan sebagai pilihan terbaik. Ashour dan Mahdiyar (2024) menyatakan bahwa proses sintesis global memungkinkan seluruh informasi yang diperoleh dari setiap tingkat hierarki digabungkan menjadi satu nilai prioritas akhir yang dapat

digunakan sebagai dasar pengambilan keputusan (Ashour & Mahdiyar, 2024). Pada program pemilihan kereta api ini, hasil sintesis global digunakan untuk menghasilkan ranking alternatif kereta sehingga pengguna dapat memilih kereta yang paling sesuai dengan preferensinya.

2.2 Skenario Kasus & Dataset

Skenario Kasus

Dalam perencanaan perjalanan menggunakan transportasi umum, calon penumpang sering kali dihadapkan pada dilema dalam menentukan pilihan kereta api yang paling optimal. Kompleksitas ini muncul karena setiap individu memiliki prioritas dan preferensi yang berbeda-beda, seperti meminimalkan anggaran (kriteria bertipe *cost*) atau memaksimalkan kenyamanan fasilitas (kriteria bertipe *benefit*).

- **Pengambil Keputusan:** Pengambil keputusan dalam skenario ini adalah seorang calon penumpang dewasa yang berencana melakukan perjalanan sendirian (1 penumpang dewasa).
- **Masalah Detail:** Penumpang perlu memilih satu opsi kereta terbaik dari berbagai alternatif jadwal yang tersedia untuk keberangkatan pada hari Minggu, 24 Mei 2026. Tantangan utamanya adalah menyeimbangkan lima kriteria yang saling bertolak belakang: biaya tiket, kenyamanan jenis rangkaian, waktu sampai di tujuan, efisiensi waktu tempuh, dan jumlah stasiun pemberhentian selama perjalanan.
- **Tujuan Akhir:** Membantu calon penumpang menentukan alternatif kereta api terbaik secara objektif dan sistematis menggunakan metode *Analytical Hierarchy Process* (AHP) berdasarkan bobot kepentingan kriteria yang telah disesuaikan dengan preferensi pribadinya.

Dataset & Sumber Data

Dataset yang digunakan dalam sistem pendukung keputusan ini merupakan data sekunder mandiri yang disusun dan diolah secara langsung. Data mentah diperoleh melalui teknik *scraping* atau pencatatan publik pada antarmuka situs resmi pemesanan

tiket PT Kereta Api Indonesia (Persero) di [booking.kai.id](https://www.kai.id). Dataset ini telah dipublikasikan di platform Kaggle dengan tautan: <https://www.kaggle.com/datasets/gianabif14/jadwal-tiket-ka-dari-stasiun-pse-24-mei-2026/>.

- Nama Dataset: Jadwal & Harga Tiket Kereta Api dari Stasiun Pasarsenen (24 Mei 2026).
- Jumlah Data: Terdiri dari 1.854 baris data.
- Cakupan Data: Informasi mencakup seluruh jadwal perjalanan kereta api (baik kursi yang masih tersedia maupun yang sudah habis terjual) dari Stasiun Pasarsenen (PSE) menuju berbagai stasiun tujuan di Pulau Jawa. Pada aplikasi sistem pendukung keputusan ini, data difokuskan secara spesifik pada rute tujuan akhir Stasiun Yogyakarta (YK) dan Stasiun Lempuyangan (LPN).
- Asumsi Perjalanan: Data dihitung untuk kapasitas 1 penumpang dewasa tanpa membawa bayi (usia < 3 tahun).

Struktur Dataset

Dataset ini memiliki struktur data terorganisasi yang memuat karakteristik kuantitatif dan kualitatif dari setiap jadwal kereta api. Atribut-atribut tersebut didefinisikan ke dalam beberapa kolom sebagai berikut:

Nama Kolom	Tipe Data	Deskripsi Atribut
Kode	String	Kode unik kombinasi antara identitas kereta, rute, dan kelas (contoh: 74B-PSE-JNG-CC).
Nama Kereta	String	Nama resmi armada kereta api.
Kode Kereta	String	Kode numerik atau huruf identifikasi kereta.
Stasiun Asal	String	Lokasi awal keberangkatan (selalu bernilai PASARSENEN).

Kode Stasiun Asal	String	Kode singkatan stasiun asal (PSE).
Stasiun Tujuan	String	Nama stasiun yang menjadi destinasi akhir perjalanan.
Kode Stasiun Tujuan	String	Kode singkatan stasiun tujuan (seperti YK, LPN, JNG, BKS, dll.).
Kelas	String	Kategori kelas perjalanan secara umum (Ekonomi, Eksekutif).
Kode Kelas	String	Sub-kelas atau kode detail tarif pelayanan (AA, AB, AC, AD, C, CA, CB, CC, CD).
Jam Berangkat	Time	Waktu keberangkatan kereta dari stasiun asal (format 24 jam).
Jam Sampai	Time	Waktu kedatangan kereta di stasiun tujuan (format 24 jam).
Durasi (menit)	Integer	Total lama waktu perjalanan yang ditempuh dalam satuan menit.
Harga (Rp)	Integer	Nominal biaya tiket perjalanan dalam satuan Rupiah.
Jumlah Stasiun Pemberhentian	Integer	Banyaknya stasiun yang disinggahi selama perjalanan, di luar stasiun asal dan stasiun tujuan.

2.3 Pemodelan Sistem Pendukung Keputusan

Pemodelan Sistem Pendukung Keputusan (SPK) dalam pemilihan kereta api ini menggunakan metode Analytical Hierarchy Process (AHP). Penilaian dilakukan dengan

menguraikan permasalahan menjadi kriteria-kriteria penentu dan alternatif pilihan yang kemudian dinilai menggunakan skala Saaty maupun transformasi data aktual.

a) Daftar Kriteria

Berdasarkan sistem yang dikembangkan, terdapat 5 kriteria utama yang digunakan untuk mengevaluasi alternatif kereta api. Kriteria tersebut dikelompokkan menjadi dua jenis tipe:

1. *Cost*: Nilai yang semakin kecil dianggap semakin baik (misalnya harga murah atau durasi perjalanan singkat).
2. *Benefit*: Nilai yang semakin besar dianggap semakin baik (misalnya kenyamanan sarana/fasilitas rangkaian).

Berikut adalah detail kriteria yang digunakan beserta pembobotan nilai dasarnya:

No	Kriteria	Kode	Tipe	Deskripsi & Nilai Aktual
1	Biaya	C1	<i>Cost</i>	Harga tiket kereta dalam satuan Rupiah (Rp).
2	Jenis Rangkaian	C2	<i>Benefit</i>	Kualitas kenyamanan armada berdasarkan modifikasi dan tipe gerbong kereta.
3	Waktu Sampai	C3	<i>Cost</i>	Waktu kedatangan di stasiun tujuan, dikonversi ke menit total sejak 00:00 (Makin awal/pagi sampai dianggap makin optimal).
4	Waktu Tempuh	C4	<i>Cost</i>	Durasi total perjalanan dari stasiun asal ke stasiun tujuan (menit).
5	Jumlah Pemberhentian	C5	<i>Cost</i>	Banyaknya stasiun singgah selama perjalanan.

b) Daftar Alternatif (Data Sampel Sederhana)

Untuk mempermudah simulasi perhitungan pada model, diambil 5 sampel alternatif dari dataset kereta dengan stasiun tujuan akhir Yogyakarta (YK) sebagai berikut:

Kode Alternatif	Nama Kereta	Kelas	C1	C2	C3	C4	C5
A1	Fajar Utama Solo	Ekonomi	Rp340.000	4	350 menit	10 menit	0
A2	Fajar Utama Solo	Eksekutif	Rp450.000	5	350 menit	10 menit	0
A3	Sawunggalih	Ekonomi	Rp150.000	2	400 menit	10 menit	0
A4	Sawunggalih	Eksekutif	Rp230.000	3	400 menit	10 menit	0
A5	Tawang Jaya Premium	Ekonomi	Rp175.000	3	416 menit	11 menit	0

Catatan Konversi Data Aktual Ke Nilai Parameter:

- Skala Penilaian Jenis Rangkaian (Benefit):

- Skor 5: *New Generation Stainless Steel* (Eksekutif)
- Skor 4: *New Generation Stainless Steel* (Ekonomi)
- Skor 3: *Eksekutif / Premium Modifikasi Mild Steel*
- Skor 2: *New Generation Modifikasi*
- Skor 1: *Rangkaian Standar / Ekonomi Biasa*

- Konversi Waktu Sampai (Cost):

Jam Sampai dikonversi ke total menit ($\text{Jam} \times 60 + \text{Menit}$) agar dapat dihitung secara matematis. Contohnya, jam 05:50 dihitung menjadi $(5 \times 60) + 50 = 350$ menit.

c) Skala Penilaian Perbandingan Berpasangan

Proses perbandingan tingkat kepentingan, baik antar kriteria maupun antar alternatif, dilakukan menggunakan Skala Intensitas Kepentingan Saaty (1–9).

Skala Nilai	Definisi Tingkat Kepentingan
1	Kedua elemen sama pentingnya (<i>Equally important</i>)
3	Elemen yang satu sedikit lebih penting dari elemen lainnya (<i>Moderately more important</i>)
5	Elemen yang satu jelas lebih penting dari elemen lainnya (<i>Strongly more important</i>)
7	Elemen yang satu sangat jelas lebih penting dari elemen lainnya (<i>Very strongly more important</i>)
9	Elemen yang satu mutlak lebih penting dari elemen lainnya (<i>Extremely more important</i>)
2, 4, 6, 8	Nilai-nilai tengah di antara dua pertimbangan yang berdekatan
Kebalikan (1/Rasio)	Jika elemen i memiliki nilai w ketika dibandingkan dengan elemen j, maka elemen j memiliki nilai 1/w ketika dibandingkan dengan elemen i

Dalam program ini, matriks perbandingan alternatif dibentuk secara otomatis oleh sistem melalui fungsi rasio nilai aktual:

Rasio = Nilai aktual j / Nilai aktual i (untuk tipe cost)

Rasio = Nilai aktual i / Nilai aktual j (untuk tipe benefit)

Jika nilai rasio berada pada rentang tertentu (misal $1,3 < \text{rasio} < 1,5$), sistem secara otomatis akan memberikan bobot skala Saaty (misal **3**) pada matriks perbandingan berpasangan alternatif tersebut.

2.4 Perhitungan & Algoritma

Secara umum, algoritma yang digunakan dalam sistem pendukung keputusan ini adalah Analytical Hierarchy Process (AHP). Algoritma ini memecahkan masalah multikriteria yang kompleks menjadi sebuah struktur hierarki yang logis. Alur komputasi keputusan ini dibagi menjadi 5 tahapan utama sebagai berikut:

- 1) Matriks Kriteria
- 2) Vektor Prioritas
- 3) Uji Konsistensi (CR)
- 4) Matriks Alternatif
- 5) Sintesis Akhir

1. Pembentukan Matriks Perbandingan Berpasangan Kriteria

Langkah pertama adalah menyusun matriks perbandingan berpasangan antar kriteria berukuran $n \times n$ (dalam kasus ini 5×5). Pengguna memasukkan nilai intensitas kepentingan berdasarkan skala Saaty (1–9). Jika elemen pada baris i lebih penting daripada kolom j dengan nilai w , maka nilai pada sel $M_{ij} = w$. Sebaliknya, nilai untuk sel kebalikannya otomatis dihitung: $M_{ji} = 1/M_{ij}$

Sedangkan untuk diagonal utama matriks ($i = j$) selalu bernilai 1 karena membandingkan kriteria dengan dirinya sendiri.

2. Normalisasi Matriks Kriteria dan Penghitungan Vektor Prioritas (Bobot)

Setelah matriks perbandingan berpasangan terisi penuh, algoritma melakukan perhitungan bobot melalui fungsi `hitung_vektor_prioritas()` dengan tahapan:

- Jumlahkan Nilai Kolom: Menghitung total nilai dari setiap kolom pada matriks kriteria.
- Normalisasi: Membagi setiap nilai sel dalam suatu kolom dengan total jumlah kolom yang bersesuaian.

- Hitung Vektor Prioritas (Bobot Kriteria): Mencari nilai rata-rata (*mean*) dari setiap baris pada matriks yang telah dinormalisasi. Hasil rata-rata baris ini merepresentasikan bobot prioritas akhir (*W*) masing-masing kriteria.

3. Pengujian Konsistensi (Consistency Check)

AHP memiliki keunggulan untuk menguji apakah penilaian yang dimasukkan pengguna bersifat logis atau kontradiktif. Algoritma menguji konsistensi dengan cara:

- Menghitung Lambda max (Eigenvalue Maksimum) dengan mengalikan matriks awal dengan vektor prioritas, lalu membagi hasilnya dengan vektor prioritas itu sendiri sebelum dirata-ratakan.
- Consistency Index (CI): Dihitung menggunakan rumus:

$$CI = \text{Lambda max} - n / n - 1$$

(di mana *n* adalah jumlah kriteria, yaitu 5).

- Consistency Ratio (CR): Membandingkan nilai CI dengan nilai *Random Index* (RI) standar Saaty (untuk *n*=5, nilai RI = 1,12).

$$CR = CI / RI$$

- Syarat Konsistensi: Jika $CR \leq 0,10$, maka matriks dinyatakan konsisten dan perhitungan dapat dilanjutkan. Jika $CR > 0,10$, pengguna harus memperbaiki input matriks perbandingan.

4. Pembentukan Matriks Perbandingan Alternatif Otomatis

Berbeda dengan kriteria yang diisi manual oleh pengguna, pada sistem ini perbandingan antar 5 alternatif kereta dilakukan secara otomatis melalui fungsi `perbandingan_berpasangan_ahp()`. Sistem membaca data aktual dari dataset dan membandingkannya secara berpasangan untuk setiap kriteria:

- Kriteria Tipe Cost (Biaya, Waktu Sampai, Waktu Tempuh): Skala perbandingan dihitung berdasarkan rasio di mana nilai yang lebih rendah mendapatkan skor Saaty yang lebih tinggi:

$$\text{Rasio} = \text{Nilai alt } j / \text{Nilai alt } i$$

- Kriteria Tipe Benefit (Jenis Rangkaian): Skala perbandingan dihitung berdasarkan rasio di mana nilai yang lebih besar mendapatkan skor Saaty yang lebih tinggi:

$$\text{Rasio} = \text{Nilai alt } i / \text{Nilai alt } j$$

Rasio ini kemudian dipetakan ke dalam skala Saaty 1-9 di dalam kode program, lalu dinormalisasi untuk menghasilkan vektor prioritas alternatif untuk masing-masing kriteria.

5. Sintesis Global (Perangkingan Akhir)

Tahap akhir dari algoritma AHP adalah melakukan perkalian matriks antara vektor prioritas kriteria dengan matriks bobot alternatif yang telah diperoleh.

Skor akhir untuk setiap alternatif A_i dihitung menggunakan rumus penjumlahan terbobot:

$$\text{Skor Akhir } A_i = \sum_{j=1}^n (W_j \times V_{ij})$$

Di mana:

- W_j = Bobot prioritas kriteria ke-j.
- V_{ij} = Bobot prioritas alternatif ke-i pada kriteria ke-j.

Alternatif kereta dengan total skor akhir tertinggi akan diurutkan di peringkat pertama sebagai rekomendasi terbaik bagi pengguna.

2.5 Implementasi & Tampilan Sistem

(Source Code Python)

```
import streamlit as st
import pandas as pd
import numpy as np
```

```

import matplotlib.pyplot as plt
from io import StringIO

# KAI Logo URL (Direct Image Link)
KAI_LOGO =
"https://upload.wikimedia.org/wikipedia/commons/5/56/Logo_PT_Kereta_Api_Indonesia_%28Persero%29_2020.svg"

st.set_page_config(
    page_title="DSS Pemilihan Kereta Api - Metode AHP",
    page_icon=KAI_LOGO,
    layout="wide",
    initial_sidebar_state="expanded"
)

st.markdown("""
<style>
@import
url('https://fonts.googleapis.com/css2?family=Inter:wght=300;400;500;600;700&display=swap');

html, body, [class*="css"] { font-family: 'Inter', sans-serif; }

.kai-header {
    background: linear-gradient(135deg, #0D2E5C 0%, #163B6E 45%, #F26522 100%);
    padding: 28px 32px;
    border-radius: 12px;
    margin-bottom: 24px;
    display: flex;
    align-items: center;
    gap: 20px;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
}

.kai-header img {
    height: 58px;
    background: rgba(255, 255, 255, 0.9);
    padding: 6px 10px;
    border-radius: 8px;

```

```

}
.kai-header-text h1 {
    color: #fff; font-size: 26px; font-weight: 700;
    margin: 0; letter-spacing: -0.3px;
    text-shadow: 1px 1px 2px rgba(0,0,0,0.2);
}
.kai-header-text p {
    color: rgba(255,255,255,0.9); font-size: 14px;
    margin: 4px 0 0 0;
}

.step-badge {
    display: inline-block;
    background: #F26522; color: #fff;
    width: 28px; height: 28px; border-radius: 50%;
    text-align: center; line-height: 28px;
    font-weight: 600; font-size: 14px; margin-right: 8px;
}
.step-badge.inactive { background: #555; }
.step-badge.done { background: #2ecc71; }

.section-title {
    font-size: 20px; font-weight: 600; color: #1a1a2e;
    border-left: 4px solid #F26522; padding-left: 12px;
    margin: 24px 0 16px 0;
}

.rank-card {
    border-radius: 10px; padding: 16px 20px;
    margin-bottom: 8px; display: flex;
    align-items: center; gap: 16px;
}
.rank-card.gold { background: linear-gradient(135deg, #F7C948, #F5A623); color: #fff; }
.rank-card.silver { background: linear-gradient(135deg, #BCC6CC, #8E9EAB); color: #fff; }
.rank-card.bronze { background: linear-gradient(135deg, #C9956B, #A0724A); color: #fff; }
.rank-card.normal { background: #f0f2f6; color: #1a1a2e; }

```

```

        .rank-number { font-size: 28px; font-weight: 700; min-width: 40px;
text-align: center; }
        .rank-name { font-size: 16px; font-weight: 600; flex: 1; }
        .rank-score { font-size: 16px; font-weight: 500; }

div[data-testid="stSidebar"] { background: #1a1a2e; }
div[data-testid="stSidebar"] * { color: #e0e0e0 !important; }
        div[data-testid="stSidebar"]      hr      {      border-color:
rgba(255,255,255,0.1) !important; }
</style>
"""', unsafe_allow_html=True)

@st.cache_data
def loadData():
    df = pd.read_csv('Data KAI PSE-YK,LPN.csv', sep=';', dtype={'Harga
(Rp)': str})
    df['Harga (Rp)'] = df['Harga (Rp)'].str.replace('.', '').astype(int)
    return df

def perbandingan_berpasangan_ahp(values, is_lower_better=True):
    n = len(values)
    matriks = np.ones((n, n))
    safe_values = [max(v, 0.1) for v in values]

    for i in range(n):
        for j in range(n):
            if i != j:
                if is_lower_better:
                    rasio = safe_values[j] / safe_values[i]
                else:
                    rasio = safe_values[i] / safe_values[j]

                if rasio >= 1:
                    if rasio <= 1.1: matriks[i, j] = 1
                    elif rasio <= 1.3: matriks[i, j] = 2
                    elif rasio <= 1.5: matriks[i, j] = 3
                    elif rasio <= 1.8: matriks[i, j] = 4
                    elif rasio <= 2.1: matriks[i, j] = 5

```

```

        elif rasio <= 2.5: matriks[i, j] = 6
        elif rasio <= 3.0: matriks[i, j] = 7
        elif rasio <= 3.5: matriks[i, j] = 8
        else: matriks[i, j] = 9
    else:
        matriks[i, j] = 1 / matriks[j, i]
return matriks

def hitung_vektor_prioritas(matriks_kriteria):
    jumlah_kolom = matriks_kriteria.sum(axis=0)
    normalisasi_kriteria = matriks_kriteria / jumlah_kolom
    bobot_kriteria = normalisasi_kriteria.mean(axis=1)
    hasil_kali = np.dot(matriks_kriteria, bobot_kriteria)
    lambda_max = np.mean(hasil_kali / bobot_kriteria)
    return bobot_kriteria, lambda_max, normalisasi_kriteria

def main():
    st.markdown(f'''
<div class="kai-header">
    
    <div class="kai-header-text">
        <h1>Sistem Pendukung Keputusan Pemilihan Kereta Api</h1>
        <p>Metode Analytical Hierarchy Process (AHP) &mdash; Rute:
Pasar Senen (PSE) &rarr; Yogyakarta (YK) / Lempuyangan (LPN)</p>
    </div>
</div>
''', unsafe_allow_html=True)

    data = loadData()

    if 'step' not in st.session_state: st.session_state.step = 1
        if 'stasiun_terpilih' not in st.session_state:
st.session_state.stasiun_terpilih = None
        if 'alternatif' not in st.session_state: st.session_state.alternatif
= []
            if 'bobot_kriteria' not in st.session_state:
st.session_state.bobot_kriteria = None
            if 'skor_akhir' not in st.session_state: st.session_state.skor_akhir
= None

```

```

# Inisialisasi awal matriks kriteria untuk data_editor Step 3
if 'matriks_berpasangan_df' not in st.session_state:
    kriteria = ["Biaya", "Jenis Rangkaian", "Waktu Sampai", "Waktu Tempuh", "Jml Pemberhentian"]
    default_matriks = np.ones((5, 5))
    st.session_state.matriks_berpasangan_df =
pd.DataFrame(default_matriks, index=kriteria, columns=kriteria)

with st.sidebar:
    st.image(KAI_LOGO, width=120)
    st.markdown("#### Langkah Analisis")

    steps = [
        "Filter Stasiun Tujuan",
        "Pilih 5 Alternatif",
        "Tentukan Bobot Kriteria",
        "Hitung dengan AHP",
        "Lihat Hasil Ranking"
    ]

    for i, step in enumerate(steps, 1):
        if i < st.session_state.step: badge = "done"
        elif i == st.session_state.step: badge = ""
        else: badge = "inactive"
        st.markdown(f'<span class="step-badge {badge}">{i}</span>
{step}', unsafe_allow_html=True)

    st.divider()
    st.caption("Metode: Analytical Hierarchy Process (AHP)")
    st.caption("© Copyright 2026. All rights reserved Kelompok 10 SCPK KAI")

    if st.session_state.step == 1: langkah1_pilih_stasiun(data)
    elif st.session_state.step == 2: langkah2_pilih_alternatif(data)
    elif st.session_state.step == 3: langkah3_bobot_kriteria()
    elif st.session_state.step == 4: langkah4_perhitungan_ahp()
    elif st.session_state.step == 5: langkah5_hasil()

```

```

def langkah1_pilih_stasiun(data):
    st.markdown('<div class="section-title">Pilih Stasiun Tujuan</div>',
unsafe_allow_html=True)
    col1, col2 = st.columns(2)
    with col1:
        if st.button("YOGYAKARTA (YK)", use_container_width=True,
type="primary"):
            st.session_state.stasiun_terpilih = "YK"
            st.session_state.step = 2
            st.rerun()
    with col2:
        if st.button("LEMPUYANGAN (LPN)", use_container_width=True,
type="primary"):
            st.session_state.stasiun_terpilih = "LPN"
            st.session_state.step = 2
            st.rerun()

    st.markdown("---")
    st.markdown('<div class="section-title">Preview Data Kereta</div>',
unsafe_allow_html=True)
    tab1, tab2 = st.tabs(["Yogyakarta (YK)", "Lempuyangan (LPN)"])

    with tab1:
        yk_trains = data[data['KodeStasiunTujuan'] ==
"YK"].drop_duplicates(subset=['Nama Kereta', 'Kelas'])
        if len(yk_trains) > 0:
            display_df = yk_trains[['Nama Kereta', 'Kelas', 'Jenis
Rangkaian', 'Harga (Rp)', 'Durasi (menit)', 'Jumlah Stasiun
Pemberhentian']].copy()
            display_df['Harga (Rp)'] = display_df['Harga
(Rp)'].apply(lambda x: f"Rp{x:,.0f}")
            st.dataframe(display_df, use_container_width=True)
        else:
            st.warning("Tidak ada data kereta ke Yogyakarta")

    with tab2:
        lpn_trains = data[data['KodeStasiunTujuan'] ==
"LPN"].drop_duplicates(subset=['Nama Kereta', 'Kelas'])
        if len(lpn_trains) > 0:

```

```

        display_df = lpn_trains[['Nama Kereta', 'Kelas', 'Jenis
Rangkaian', 'Harga (Rp)', 'Durasi (menit)', 'Jumlah Stasiun
Pemberhentian']].copy()

        display_df['Harga (Rp)'] = display_df['Harga
(Rp)'].apply(lambda x: f"Rp{x:,.0f}")

        st.dataframe(display_df, use_container_width=True)
    else:
        st.warning("Tidak ada data kereta ke Lempuyangan")

def langkah2_pilih_alternatif(data):
    st.markdown('<div class="section-title">Pilih Kelas & Alternatif
Kereta</div>', unsafe_allow_html=True)

    base_filtered_df = data[data['KodeStasiunTujuan'] ==
st.session_state.stasiun_terpilih].drop_duplicates(subset=['Nama
Kereta', 'Kelas'])

    st.info(f"Stasiun tujuan: **{st.session_state.stasiun_terpilih}**")

    kelas_options = ["Semua Kelas"] +
sorted(list(base_filtered_df['Kelas'].unique()))

    selected_kelas = st.radio("Filter Kelas Kereta:", kelas_options,
horizontal=True)

    filtered_df = base_filtered_df if selected_kelas == "Semua Kelas"
else base_filtered_df[base_filtered_df['Kelas'] == selected_kelas]

    st.caption(f"Tersedia {len(filtered_df)} pilihan kereta. Silakan
pilih tepat 5 kereta.")

    daftar_alternatif = []
    for idx, row in filtered_df.iterrows():
        alt_text = f"{row['Nama Kereta']} | {row['Kelas']} | Rp{row['Harga
(Rp)']:,.0f} | {row['Durasi (menit)']} mnt | {row['Jumlah Stasiun
Pemberhentian']} st | {row['Jam Sampai']} | {row['Jenis Rangkaian']}"
        jam, menit = map(int, str(row['Jam Sampai']).split(':'))
        waktu_sampai = jam * 60 + menit

        r = str(row['Jenis Rangkaian']).lower()
        if 'new generasion' in r and 'eksekutif' in r: rangkaian_score =
5

        elif 'new generasion' in r: rangkaian_score = 4

```



```

elif 'eksekutif' in r or 'premium' in r: rangkaian_score = 3
elif 'modifikasi' in r: rangkaian_score = 2
else: rangkaian_score = 1

daftar_alternatif.append({
    'key': idx, 'text': alt_text, 'nama': row['Nama Kereta'],
'kelas': row['Kelas'],
    'harga': row['Harga (Rp)'], 'durasi': row['Durasi (menit)'],
'stasiun': row['Jumlah Stasiun Pemberhentian'],
    'rangkaian': rangkaian_score, 'waktu_sampai': waktu_sampai,
'rangkaian_nama': row['Jenis Rangkaian'], 'jam_sampai': row['Jam
Sampai']
})

dipilih = st.multiselect("Pilih 5 alternatif kereta:",
options=[alt['text'] for alt in daftar_alternatif])
col1, col2 = st.columns(2)

with col1:
    if len(dipilih) == 5:
        if st.button("Simpan Pilihan", use_container_width=True,
type="primary"):
            st.session_state.alternatif = [alt for sel in dipilih
for alt in daftar_alternatif if alt['text'] == sel]
            st.session_state.step = 3
            st.rerun()
    else:
        st.button("Simpan Pilihan", disabled=True,
use_container_width=True)
    with col2:
        if st.button("🔄 Reset Pilihan", use_container_width=True):
st.rerun()

st.markdown("---")
st.metric("Jumlah Terpilih", f"{len(dipilih)} / 5")
if dipilih:
    for i, sel in enumerate(dipilih, 1): st.success(f"{i}. {sel}")

def langkah3_bobot_kriteria():

```

```

st.markdown('<div class="section-title">Tentukan Bobot Kriteria
(Matriks Berpasangan Fleksibel)</div>', unsafe_allow_html=True)

st.info("""
💡 Cara Pengisian:
1. Isi langsung nilai tingkat kepentingan kriteria pada tabel di
bawah (Skala Saaty: 1 sampai 9).
2. Jika Baris lebih penting dari Kolom, masukkan angka 2 s/d
9 (Misal Biaya vs Jenis Rangkaian = 3).
3. Jika Kolom lebih penting dari Baris, masukkan pecahan
desimalnya (Misal 0.33, 0.5, dll).
4. Sistem otomatis menghitung sel kebalikannya secara real-time (1
/ nilai input).
5. DIAGONAL UTAMA TERKUNCI MUTLAK: Perbandingan kriteria dengan
dirinya sendiri (diagonal utama) wajib bernilai 1.0. Jika Anda
mencoba mengubahnya, nilai otomatis dibatalkan & dipaksa kembali ke
1.0.
""")

kriteria = ["Biaya", "Jenis Rangkaian", "Waktu Sampai", "Waktu
Tempuh", "Jml Pemberhentian"]
df_saat_ini = st.session_state.matriks_berpasangan_df.copy()

st.markdown("### 📊 Tabel Matriks Perbandingan Berpasangan")

df_diedit = st.data_editor(
    df_saat_ini,
    disabled=["Index"],
    num_rows="fixed",
    column_config={
        c: st.column_config.NumberColumn(min_value=0.01,
max_value=9.0, format="%.3f") for c in kriteria
    },
    use_container_width=True
)

matriks_diubah = False
reset_diagonal = False

```

```

for i in range(len(kriteria)):
    for j in range(len(kriteria)):
        if i == j:
            if df_diedit.iloc[i, j] != 1.0:
                df_diedit.iloc[i, j] = 1.0
                reset_diagonal = True
                matriks_diubah = True
        else:
            if df_diedit.iloc[i, j] != df_saas_ini.iloc[i, j]:
                val = df_diedit.iloc[i, j]
                if val <= 0: val = 0.01
                df_diedit.iloc[j, i] = round(1 / val, 4)
                matriks_diubah = True
            elif df_diedit.iloc[j, i] != df_saas_ini.iloc[j, i]:
                val = df_diedit.iloc[j, i]
                if val <= 0: val = 0.01
                df_diedit.iloc[i, j] = round(1 / val, 4)
                matriks_diubah = True

if matriks_diubah:
    st.session_state.matriks_berpasangan_df = df_diedit
    if reset_diagonal:
        st.toast("⚠ Perbandingan dengan kriteria sendiri dikunci mutlak sebesar 1.0!", icon="🚫")
    st.rerun()

matriks = df_diedit.to_numpy()
columns1, columns2 = st.columns([1, 1])

with columns1:
    if st.button("Hitung Bobot Kriteria & Konsistensi", use_container_width=True, type="primary"):
        bobot, lambda_max, matriks_ternormalisasi = hitung_vektor_prioritas(matriks)
        n = len(kriteria)
        ci = (lambda_max - n) / (n - 1) if n > 1 else 0
        ri = [0, 0, 0.58, 0.9, 1.12, 1.24, 1.32, 1.41, 1.45, 1.49]
        cr = ci / ri[n-1] if ri[n-1] != 0 else 0

```

```

        st.session_state.bobot_kriteria = {
            'bobot': bobot, 'kriteria': kriteria, 'cr': cr, 'matriks':
matriks,
            'matriks_ternormalisasi': matriks_ternormalisasi,
'lambda_max': lambda_max, 'ci': ci
        }
        with columns2:
            if st.button("🔄 Reset Tabel ke Default (1.0)",
use_container_width=True):
                st.session_state.matriks_berpasangan_df =
pd.DataFrame(np.ones((5, 5)), index=kriteria, columns=kriteria)
                st.session_state.bobot_kriteria = None
                st.rerun()

            if st.session_state.bobot_kriteria is not None:
                hasil_bobot = st.session_state.bobot_kriteria
                st.markdown('<div class="section-title">Hasil Analisis Bobot
Kriteria</div>', unsafe_allow_html=True)

                # Menampilkan tabel normalisasi & bobot seperti kode awal
                st.markdown('##### Matriks Normalisasi & Bobot (W)')
                norm_df = pd.DataFrame(hasil_bobot['matriks_ternormalisasi'],
index=kriteria, columns=kriteria)
                norm_df['Bobot (W)'] = hasil_bobot['bobot']
                st.dataframe(norm_df.style.format("{:.4f}"),
use_container_width=True)

                c1, c2, c3 = st.columns(3)
                c1.metric("Lambda Max ( $\lambda$  max)",
f"{round(hasil_bobot['lambda_max'], 4)}")
                c2.metric("Consistency Index (CI)", f"{round(hasil_bobot['ci'],
4)}")
                c3.metric("Consistency rasio (CR)", f"{round(hasil_bobot['cr'],
4)}")

                if hasil_bobot['cr'] <= 0.1:
                    st.success("✅ Matriks Konsisten! ( $CR \leq 0.1$ ). Silakan lanjut
ke langkah berikutnya.")

```

```

        if st.button("Lanjut ke Perhitungan AHP Silakan sesuaikan kembali nilai di tabel di atas agar logis.")

def langkah4_perhitungan_ahp():
    """Step 4: Mengembalikan detail matriks perbandingan alternatif per
    kriteria seperti file awal"""
    st.markdown('<div class="section-title">Perhitungan AHP (Matriks  
Ternilai Alternatif per Kriteria)</div>', unsafe_allow_html=True)
    if st.session_state.bobot_kriteria is None or not
st.session_state.alternatif:
        st.error("Lengkapi langkah sebelumnya!"); return

    daftar_alternatif = st.session_state.alternatif
    bobot_kriteria = st.session_state.bobot_kriteria['bobot']

    st.info(f"Analisis Berpasangan untuk {len(daftar_alternatif)}  
alternatif pilihan ke stasiun {st.session_state.stasiun_terpilih}")

    nilai_harga = [alt['harga'] for alt in daftar_alternatif]
    nilai_rangkaian = [alt['rangkaian'] for alt in daftar_alternatif]
    nilai_waktu_sampai = [alt['waktu_sampai'] for alt in
daftar_alternatif]
    nilai_durasi = [alt['durasi'] for alt in daftar_alternatif]
    nilai_stasiun = [alt['stasiun'] for alt in daftar_alternatif]
    nama_alternatif = [f"{alt['nama']} ({alt['kelas']})" for alt in
daftar_alternatif]

    # 1. Biaya (Harga)
    st.markdown("### 1. Perbandingan Berdasarkan Biaya (Harga)")
    harga_matriks = perbandingan_berpasangan_ahp(nilai_harga,
is_lower_better=True)
    bobot_harga, _, harga_norm = hitung_vektor_prioritas(harga_matriks)
    harga_df = pd.DataFrame(harga_matriks, index=nama_alternatif,
columns=nama_alternatif)

```

```

        st.dataframe(harga_df.style.format("{:.2f}"),
use_container_width=True)

    with st.expander("Lihat Matriks Normalisasi & Bobot (W) - Biaya"):
        harga_norm_df = pd.DataFrame(harga_norm, index=nama_alternatif,
columns=nama_alternatif)
        harga_norm_df['Bobot (W)'] = bobot_harga
        st.dataframe(harga_norm_df.style.format("{:.4f}"),
use_container_width=True)

# 2. Jenis Rangkaian
st.markdown("### 2. Perbandingan Berdasarkan Jenis Rangkaian")
rangkaian_matriks = perbandingan_berpasangan_ahp(nilai_rangkaian,
is_lower_better=False)
        bobot_rangkaian, _, rangkaian_norm =
hitung_vektor_prioritas(rangkaian_matriks)
        rangkaian_df = pd.DataFrame(rangkaian_matriks,
index=nama_alternatif, columns=nama_alternatif)
        st.dataframe(rangkaian_df.style.format("{:.2f}"),
use_container_width=True)
    with st.expander("Lihat Matriks Normalisasi & Bobot (W) - Jenis
Rangkaian"):
        rangkaian_norm_df = pd.DataFrame(rangkaian_norm,
index=nama_alternatif, columns=nama_alternatif)
        rangkaian_norm_df['Bobot (W)'] = bobot_rangkaian
        st.dataframe(rangkaian_norm_df.style.format("{:.4f}"),
use_container_width=True)

# 3. Waktu Sampai
st.markdown("### 3. Perbandingan Berdasarkan Waktu Sampai")
        waktu_sampai_matriks =
perbandingan_berpasangan_ahp(nilai_waktu_sampai, is_lower_better=True)
        bobot_waktu_sampai, _, ws_norm =
hitung_vektor_prioritas(waktu_sampai_matriks)
        ws_df = pd.DataFrame(waktu_sampai_matriks, index=nama_alternatif,
columns=nama_alternatif)
        st.dataframe(ws_df.style.format("{:.2f}"), use_container_width=True)
    with st.expander("Lihat Matriks Normalisasi & Bobot (W) - Waktu
Sampai"):

```

```

ws_norm_df = pd.DataFrame(ws_norm, index=nama_alternatif,
columns=nama_alternatif)

ws_norm_df['Bobot (W)'] = bobot_waktu_sampai
st.dataframe(ws_norm_df.style.format("{:.4f}"),
use_container_width=True)

# 4. Waktu Tempuh
st.markdown("### 4. Perbandingan Berdasarkan Waktu Tempuh")
durasi_matriks = perbandingan_berpasangan_ahp(nilai_durasi,
is_lower_better=True)

bobot_durasi, _, durasi_norm =
hitung_vektor_prioritas(durasi_matriks)
durasi_df = pd.DataFrame(durasi_matriks, index=nama_alternatif,
columns=nama_alternatif)
st.dataframe(durasi_df.style.format("{:.2f}"),
use_container_width=True)

with st.expander("Lihat Matriks Normalisasi & Bobot (W) - Waktu
Tempuh"):
durasi_norm_df = pd.DataFrame(durasi_norm, index=nama_alternatif,
columns=nama_alternatif)
durasi_norm_df['Bobot (W)'] = bobot_durasi
st.dataframe(durasi_norm_df.style.format("{:.4f}"),
use_container_width=True)

# 5. Jumlah Pemberhentian
st.markdown("### 5. Perbandingan Berdasarkan Jumlah Pemberhentian")
stasiun_matriks = perbandingan_berpasangan_ahp(nilai_stasiun,
is_lower_better=True)

bobot_stasiun, _, stasiun_norm =
hitung_vektor_prioritas(stasiun_matriks)
stasiun_df = pd.DataFrame(stasiun_matriks, index=nama_alternatif,
columns=nama_alternatif)
st.dataframe(stasiun_df.style.format("{:.2f}"),
use_container_width=True)

with st.expander("Lihat Matriks Normalisasi & Bobot (W) - Jumlah
Pemberhentian"):
stasiun_norm_df = pd.DataFrame(stasiun_norm,
index=nama_alternatif, columns=nama_alternatif)
stasiun_norm_df['Bobot (W)'] = bobot_stasiun

```

```

        st.dataframe(stasiun_norm_df.style.format("{:.4f}"),
use_container_width=True)

# Ringkasan Prioritas Alternatif per Kriteria
    st.markdown('<div class="section-title">Prioritas Alternatif per
Kriteria (Summary)</div>', unsafe_allow_html=True)
    prioritas_data = []
    for i, alt in enumerate(daftar_alternatif):
        prioritas_data.append({
            'Alternatif': f"{{alt['nama']}} ({{alt['kelas']}})",
            'Biaya': f"{{bobot_harga[i]:.4f}}",
            'Jenis Rangkaian': f"{{bobot_rangkaian[i]:.4f}}",
            'Waktu Sampai': f"{{bobot_waktu_sampai[i]:.4f}}",
            'Waktu Tempuh': f"{{bobot_durasi[i]:.4f}}",
            'Jml Pemberhentian': f"{{bobot_stasiun[i]:.4f}}
        })
    st.dataframe(pd.DataFrame(prioritas_data), use_container_width=True)

# Sintesis Global / Hitung Skor Akhir
    st.markdown('<div class="section-title">Sintesis Global (Skor
Akhir)</div>', unsafe_allow_html=True)
    daftar_skor = []
    for i, alt in enumerate(daftar_alternatif):
        skor = (bobot_kriteria[0] * bobot_harga[i] +
                bobot_kriteria[1] * bobot_rangkaian[i] +
                bobot_kriteria[2] * bobot_waktu_sampai[i] +
                bobot_kriteria[3] * bobot_durasi[i] +
                bobot_kriteria[4] * bobot_stasiun[i])

        detail_teks = (f"({bobot_kriteria[0]:.4f} × {{bobot_harga[i]:.4f}})
+ "
                        f"({bobot_kriteria[1]:.4f} ×
{{bobot_rangkaian[i]:.4f}}) + "
                        f"({bobot_kriteria[2]:.4f} ×
{{bobot_waktu_sampai[i]:.4f}}) + "
                        f"({bobot_kriteria[3]:.4f} × {{bobot_durasi[i]:.4f}})
+ "
                        f"({bobot_kriteria[4]:.4f} ×
{{bobot_stasiun[i]:.4f}})")

```



```

    daftar_skor.append({
        'Alternatif': f"{alt['nama']} ({alt['kelas']})",
        'Skor': skor,
        'Detail': detail_teks
    })

    final_df = pd.DataFrame(daftar_skor).sort_values('Skor',
ascending=False)

    show_detail = st.checkbox("Tampilkan Detail Rumus Perhitungan
Sintesis")
    if show_detail:
        display_df = final_df.copy()
        display_df['Skor'] = display_df['Skor'].apply(lambda x:
f"{x:.4f}")
        st.dataframe(display_df.rename(columns={'Detail': 'Detail (Σ
Bobot Kriteria × Bobot Alternatif)'}), use_container_width=True)
    else:
        st.dataframe(final_df[['Alternatif',
'Skor']].style.format({"Skor": "{:.4f}"}), use_container_width=True)

    st.session_state.skor_akhir = daftar_skor

    columns1, columns2 = st.columns(2)
    with columns1:
        if st.button("Lihat Hasil Ranking ➡", use_container_width=True,
type="primary"):
            st.session_state.step = 5
            st.rerun()
    with columns2:
        if st.button("← Kembali ke Bobot Kriteria",
use_container_width=True):
            st.session_state.step = 3
            st.rerun()

def langkah5_hasil():
    st.markdown('<div class="section-title">Hasil Ranking
Alternatif</div>', unsafe_allow_html=True)

```

```

    if st.session_state.skor_akhir is None: st.error("Hitung AHP dulu!");
    return

    sorted_scores = sorted(st.session_state.skor_akhir, key=lambda x:
x['Skor'], reverse=True)
    rank_classes = ['gold', 'silver', 'bronze']
    for i, score in enumerate(sorted_scores):
        cls = rank_classes[i] if i < 3 else 'normal'
        st.markdown(f'<div class="rank-card {cls}"><div class="rank-
number">#{i+1}</div><div class="rank-
name">{score["Alternatif"]}</div><div class="rank-score">Skor:
{score["Skor"]:.4f}</div></div>', unsafe_allow_html=True)
        with st.expander(f"Detail Perhitungan #{i+1} -
{score['Alternatif']}"):
            st.code(score['Detail'], language="text")

    st.markdown('<div class="section-title">Visualisasi Perbandingan
Skor</div>', unsafe_allow_html=True)
    fig, ax = plt.subplots(figsize=(10, 4))
    fig.patch.set_facecolor('#0E1117'); ax.set_facecolor('#0E1117')
    names = [s['Alternatif'] for s in sorted_scores]
    scores = [s['Skor'] for s in sorted_scores]
    bars = ax.barh(names, scores, color=['#F5A623', '#8E9EAB', '#A0724A']
+ ['#F26522']*(len(scores)-3))
    ax.invert_yaxis(); ax.tick_params(colors='#e0e0e0')

    for bar, s in zip(bars, scores):
        ax.text(bar.get_width() + 0.005, bar.get_y() +
bar.get_height()/2,
        f'{s:.4f}', va='center', fontsize=10, color='#e0e0e0')

    st.pyplot(fig)

    st.markdown("---")
    st.markdown('<div class="section-title">Rekomendasi</div>',
unsafe_allow_html=True)
    st.success(f"***Berdasarkan perhitungan AHP, rekomendasi terbaik
adalah:** \n\n **{sorted_scores[0]['Alternatif']}** dengan skor
**{sorted_scores[0]['Skor']:.4f}**")

```

```

columns1, columns2 = st.columns(2)
with columns1:
    if st.button("Mulai Lagi dari Awal 🔄", use_container_width=True):
        for k in ['step', 'stasiun_terpilih', 'alternatif',
'bobot_kriteria', 'skor_akhir', 'matriks_berpasangan_df']:
            if k in st.session_state: del st.session_state[k]
            st.rerun()
with columns2:
    if st.button("← Kembali ke Perhitungan",
use_container_width=True):
        st.session_state.step = 4
        st.rerun()

if __name__ == "__main__":
    main()

```

(Gambar 2.1 Tampilan Antarmuka Filter Stasiun Tujuan dan *Preview* Dataset Kereta)

KAI Sistem Pendukung Keputusan Pemilihan Kereta Api
Metode Analytical Hierarchy Process (AHP) – Rule: Pair-Score (PSE) – Yogyakarta (YK) / Lempungan (LPN)

Pilih Stasiun Tujuan

YOGYAKARTA (YK) LEMPUANGAN (LPN)

Preview Data Kereta
Yogyakarta (YK) Lempungan (LPN)

No	Nama Kereta	Kelas	Jenis Rangkaian	Harga (Rp)	Durasi (menit)	Jumlah Stasiun Pemberhentian
1745	KALAJA UTAMA SOLO	Ekonomi	New Generation Stainless Steel	Rp410,000	414	11
1746	KALAJA UTAMA SOLO	Eksekutif	New Generation Stainless Steel	Rp510,000	414	11
1747	KALAJA UTAMA YK	Ekonomi	New Generation Stainless Steel	Rp310,000	455	13
1748	KALAJA UTAMA YK	Eksekutif	New Generation Stainless Steel	Rp410,000	455	13
1749	KA TAMBAHAN PSE LPN	Eksekutif	Eksekutif Mild Steel	Rp440,000	443	8
1750	BOGOWONTO	Eksekutif	New Generation Stainless Steel	Rp410,000	448	13
1751	BOGOWONTO	Ekonomi	Premium Stainless Steel	Rp310,000	448	13
1752	SENJA UTAMA YK	Ekonomi	New Generation Stainless Steel	Rp310,000	420	11
1753	SENJA UTAMA YK	Eksekutif	New Generation Stainless Steel	Rp410,000	420	11
1754	MADJUN JARA	Eksekutif	Eksekutif New Generation Modified Mild Steel	Rp510,000	415	7

(Gambar 2.2 Tampilan Memilih Alternatif Jadwal Kereta Api Berdasarkan Filter Kelas)

KAI Sistem Pendukung Keputusan Pemilihan Kereta Api
Metode Analytical Hierarchy Process (AHP) – Rule: Pair-Seri (PS) + Triangular (TN) / Lemparangan (LPN)

Pilih Kelas dan Alternatif Kereta

Stasiun tujuan: **YK**

Filter Kelas Kereta:
☐ Semua Kelas ☒ Ekonomi ☐ Eksekutif

Terdapat 7 pilihan kereta. Silakan pilih input 3 kereta.

Pilih 3 alternatif kereta:
FAJAR UTAMA S... **SENJA UTAMA Y...** **BOGOWONTO** **E...** **MATARAM** **T...** **FAJAR UTAMA YK...**

Sempit Pilihan Reset Pilihan

Jumlah Terpilih: 5 / 5

1. FAJAR UTAMA SOLO | Ekonomi | Rp450,000 | 434 mm | 11 s | 12-34 | New Generation Stainless Steel
2. SENJA UTAMA YK | Ekonomi | Rp330,000 | 420 mm | 11 s | 02-06 | New Generation Stainless Steel
3. BOGOWONTO | Ekonomi | Rp330,000 | 440 mm | 13 s | 01-38 | Premium Stainless Steel

(Gambar 2.3 Tampilan Input Matriks Perbandingan Berpasangan Antarkriteria)

KAI Sistem Pendukung Keputusan Pemilihan Kereta Api
Metode Analytical Hierarchy Process (AHP) – Rule: Pair-Seri (PS) + Triangular (TN) / Lemparangan (LPN)

Tentukan Bobot Kriteria (Matriks Berpasangan Fleksibel)

Cara Pengisian:

1. Isi langsung nilai tingkat kepentingan kriteria pada tabel di bawah (Skala Saaty: 1 sampai 9).
2. Jika Anda lebih penting dari kolom, masukkan angka 2 s.d 9 (nilai Baras vs Jenis Rangkaian = 3).
3. Jika kolom lebih penting dari Baris, masukkan pecahan desimalnya (Misal 0,33, 0,5, dll).
4. Sistem otomatis menghitung set kebalikannya secara real time (1 / nilai diinput).
5. DIAGONAL UTAMA TERKUNCI MUTLAK: Perbandingan kriteria dengan dirinya sendiri (diagonal utama) wajib bernilai 1,0. Jika Anda mencoba mengubahnya, nilai otomatis dibatalkan & dipaksa kembali ke 1,0.

Tabel Matriks Perbandingan Berpasangan

P_j	P_1 Waktu	P_2 Jenis Rangkaian	P_3 Waktu Sampai	P_4 Waktu Tempuh	P_5 Jenis Pemberhentian
P_1 Waktu	1,000	3,000	5,000	4,000	6,000
P_2 Jenis Rangkaian	0,333	1,000	3,000	2,000	4,000
P_3 Waktu Sampai	0,200	0,333	1,000	0,500	3,000
P_4 Waktu Tempuh	0,250	0,500	2,000	1,000	3,000
P_5 Jenis Pemberhentian	0,167	0,250	0,333	0,500	1,000

Hitung Bobot Kriteria & Konsistensi Reset Tabel ke Default (1.0)

Hasil Analisis Bobot Kriteria

Matriks Normalisasi & Bobot (W)

	Waktu	Jenis Rangkaian	Waktu Sampai	Waktu Tempuh	Jenis Pemberhentian	Bobot (W)
Waktu	0,3333	0,0909	0,4348	0,5196	0,3750	0,4887

(Gambar 2.4 Tampilan Matriks Normalisasi, Nilai Bobot Kriteria, dan Uji Konsistensi CR)

KAI Sistem Pendukung Keputusan Pemilihan Kereta Api
Metode Analytical Hierarchy Process (AHP) – Rule: Pair-Seri (PS) + Triangular (TN) / Lemparangan (LPN)

Hasil Analisis Bobot Kriteria

Matriks Normalisasi & Bobot (W)

	Waktu	Jenis Rangkaian	Waktu Sampai	Waktu Tempuh	Jenis Pemberhentian	Bobot (W)
Waktu	0,3333	0,0909	0,4348	0,5196	0,3750	0,4887
Jenis Rangkaian	0,1778	0,2909	0,2609	0,2593	0,2500	0,2208
Waktu Sampai	0,1000	0,0606	0,0670	0,0638	0,1250	0,0888
Waktu Tempuh	0,1282	0,0909	0,1778	0,1277	0,1875	0,1433
Jenis Pemberhentian	0,0855	0,0402	0,0405	0,0425	0,0625	0,0556

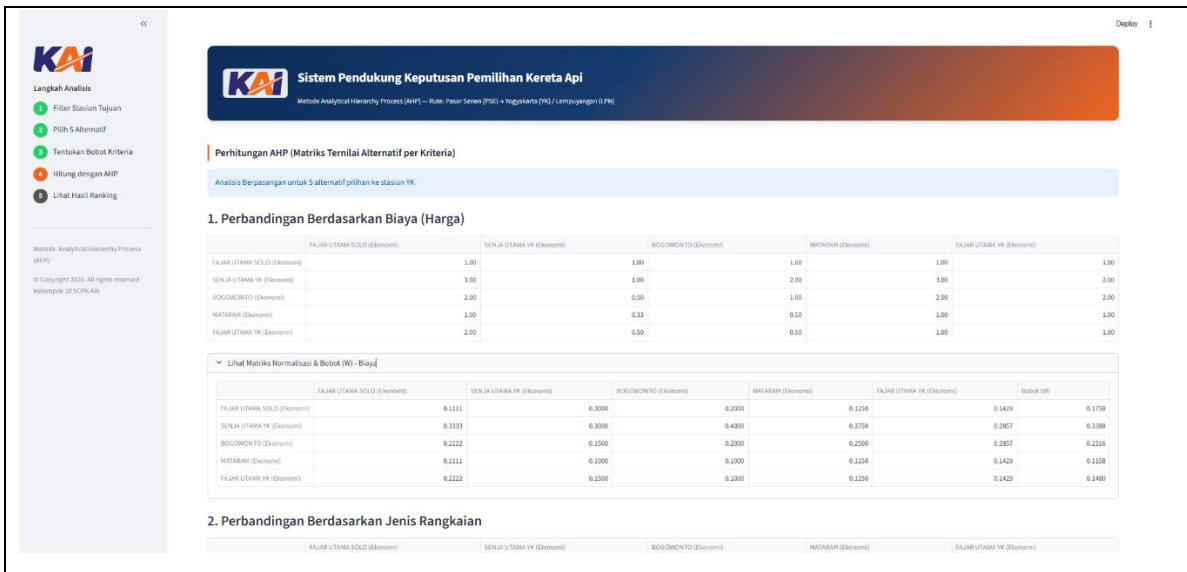
Consistency Index (CI) 0.0248

Consistency Ratio (CR) 0.0221

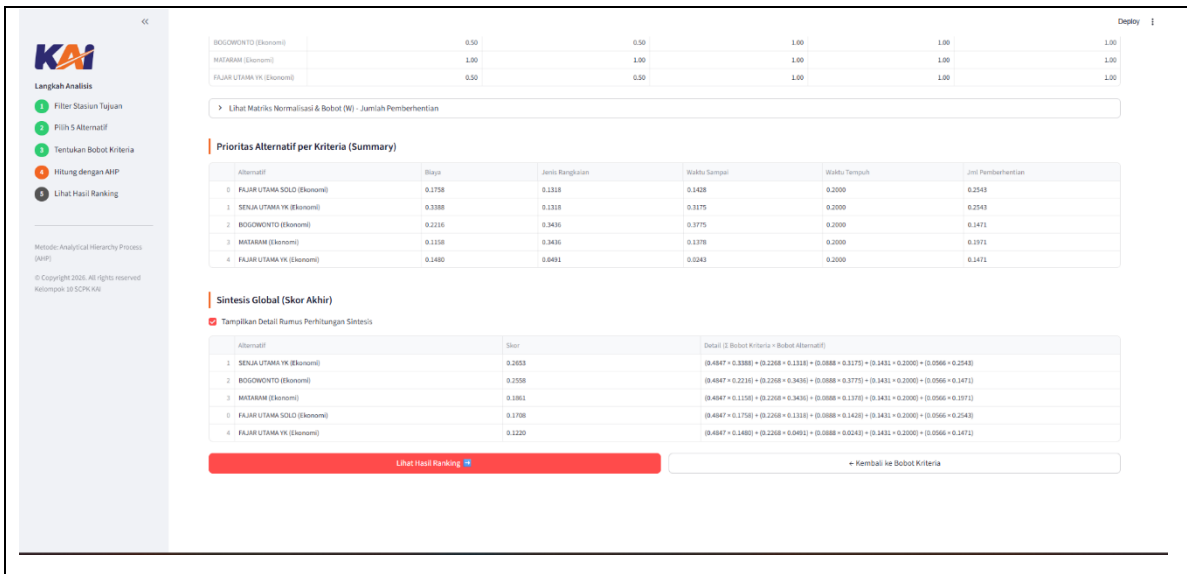
Matriks Konsisten (CR < 0.1). Silakan lanjut ke langkah berikutnya.

Lanjut ke Perhitungan AHP

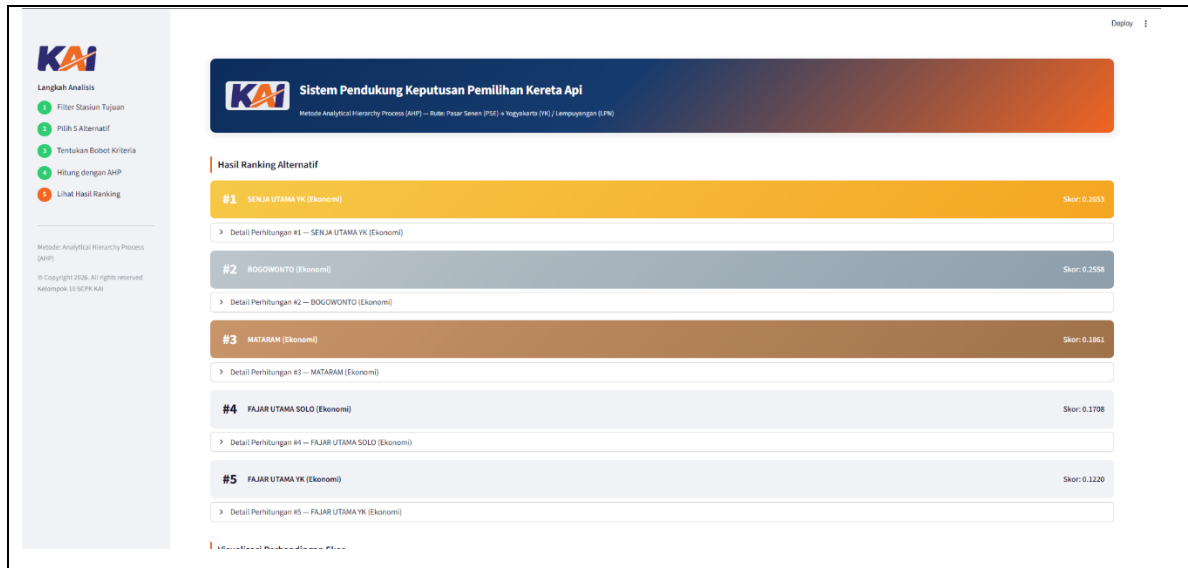
(Gambar 2.5 Tampilan Perhitungan Matriks Perbandingan Berpasangan Alternatif per Kriteria)



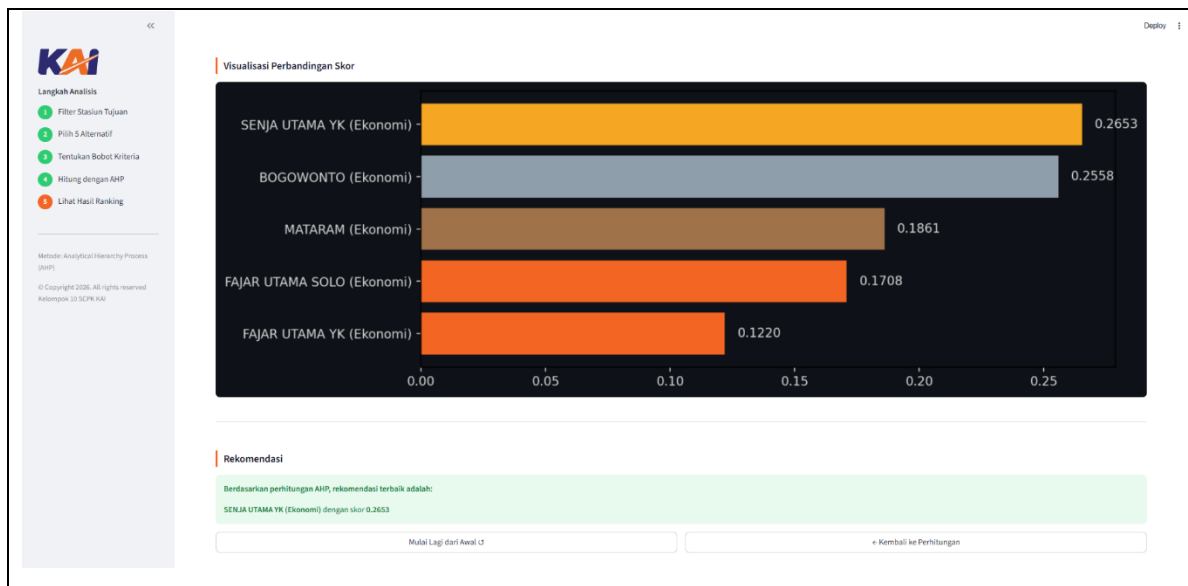
(Gambar 2.6 Tampilan Tabel Ringkasan Prioritas Alternatif dan Rumus Sintesis Global Skor Akhir)



(Gambar 2.7 Tampilan Hasil Akhir Perankingan Alternatif Kereta Api)



(Gambar 2.8 Tampilan Grafik Visualisasi Perbandingan Skor Akhir dan Blok Rekomendasi Sistem)



BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	MEI 2026			
		Minggu			
		1	2	3	4
1	Menentukan Permodelan dan Menentukan Bagaimana Logika Program				
2	Mencari Data DiAccess by KAI, Membuat Dataset dari 0 dan Upload Dataset ke Kaggle				
3	Membuat Program Python dan Mengimplementasikan Metode AHP Kedalam Program				
4	Membuat Tampilan GUI Agar Bagus dan Mudah Digunakan Oleh User				
5	Melakukan Evaluasi Pada Program Yang Telah Dibuat				
6	Membuat Laporan Project Akhir				

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Fahri Hidayatullah	123240002	- Menentukan logika program dan bagaimana cara program bekerja

			- Membuat Tampilan GUI supaya enak dilihat dan mudah dipahami - Membantu Gian dalam pembuatan dataset - Membuat Laporan Project Akhir
2	Gian Abi Firdaus	123240017	- Menentukan logika program dan bagaimana cara program bekerja - Membuat Dataset dari 0, berdasarkan data asli dari Access by KAI - Membuat perhitungan pada program agar sesuai dengan ketentuan - Melakukan uji coba program agar berjalan dengan lancar

BAB IV

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan seluruh tahapan simulasi perhitungan yang telah dilakukan pada proyek akhir praktikum Sistem Cerdas dan Pendukung Keputusan ini, dapat disimpulkan bahwa Sistem Pendukung Keputusan (SPK) pemilihan jadwal kereta api terbaik rute Stasiun Pasar Senen menuju Yogyakarta telah berhasil dibangun dan berfungsi dengan baik menggunakan *framework* Streamlit. Sistem ini terbukti mampu mengintegrasikan aspek kualitatif dan kuantitatif dari data aktual perjalanan—seperti harga tiket, jenis rangkaian, durasi, dan stasiun pemberhentian—ke dalam sebuah antarmuka yang interaktif dan mudah dipahami oleh pengguna.

Melalui penerapan metode *Analytical Hierarchy Process* (AHP), proses pengambilan keputusan yang semula bersifat subjektif dan membingungkan dapat diselesaikan secara lebih objektif dan terstruktur. Adanya fitur pengujian konsistensi (*Consistency Ratio*) di dalam sistem juga memastikan bahwa penilaian atau bobot kepentingan yang dimasukkan oleh calon penumpang tetap logis dan tidak kontradiktif sebelum matriks perhitungan diproses. Pada akhirnya, sistem ini berhasil memberikan rekomendasi alternatif jadwal kereta api yang paling optimal secara cepat dan akurat, yang diurutkan berdasarkan total skor tertinggi sesuai dengan preferensi dan skala prioritas unik dari masing-masing pengguna.

4.2 Saran

Meskipun sistem ini sudah dapat berfungsi dengan baik sesuai skenario kasus, terdapat beberapa saran yang dapat digunakan sebagai acuan untuk pengembangan dan perluasan sistem di masa mendatang:

1. Integrasi Data secara Real-Time: Pengembangan selanjutnya diharapkan tidak lagi menggunakan data sekunder statis (*dataset scraping*), melainkan dapat terintegrasi langsung dengan API resmi PT Kereta Api Indonesia (Persero) atau *platform* pihak

ketiga agar ketersediaan kursi (*seat availability*) dan harga tiket terpantau secara *real-time*.

2. Penambahan Fitur Profiling Pengguna: Sistem dapat dikembangkan dengan menambahkan fitur simpan preferensi atau profil pengguna (misalnya: profil *backpacker* yang mengutamakan *cost*, atau profil keluarga yang mengutamakan kenyamanan rangkaian), sehingga proses input matriks kriteria tidak perlu dilakukan secara manual dari awal setiap kali aplikasi digunakan.
3. Perluasan Cakupan Rute dan Transportasi: Sistem rekomendasi ini dapat diperluas cakupannya tidak hanya untuk rute Jakarta-Yogyakarta saja, tetapi juga mencakup rute-rute padat lainnya di Pulau Jawa, atau dikembangkan menjadi sistem intermoda yang membandingkan moda transportasi lain seperti bus dan pesawat.

DAFTAR PUSTAKA

- Ashour, M., & Mahdiyar, A. (2024). *A Comprehensive State-of-the-Art Survey on the Recent Modified and Hybrid Analytic Hierarchy Process Approaches*. *Applied Soft Computing*, 150, 111014.
- Kovačić, M., Vranješ, M., Barić, D., & Babić, D. (2024). *Analytic Hierarchy Process in Transportation Decision-Making: A Two-Staged Review on the Themes and Trends of Two Decades*. *Expert Systems with Applications*, 125491.
- Madzík, P., & Falát, L. (2022). *State-of-the-Art on Analytic Hierarchy Process in the Last 40 Years: Literature Review Based on Latent Dirichlet Allocation Topic Modelling*. *PLOS ONE*, 17(5), e0268777.
- Kumar, A., & Pant, S. (2022). *Analytical Hierarchy Process for Sustainable Agriculture: An Overview*. *MethodsX*, 10, 101954.
- Salehzadeh, R., & Ziaeeian, M. (2024). *Decision Making in Human Resource Management: A Systematic Review of the Applications of Analytic Hierarchy Process*. *Frontiers in Psychology*, 15, 1400772.